1 Intervalos en haskell

Los intervalos en haskell son una herramienta que permite generar listas de elementos de manera automatica, estableciendo un valor inicial, uno final y, cuando sea necesario, un patrón de incremento o decremento. Se utilizan principalmente para trabajar con secuencias de números y caracteres sin tener que escribir cada elemento individualmente.

Una de sus caracteristicas es que permiten crear listas finitas e infintas. Además, facilitan la programación al reducir la cantidad de código necesario para generar consecuencias. Los intervalos son especialmente utiles cuando se necesita recorrer valores, realizar calculos repetitivos o trabajar con conjuntos de datos ordenados.

Ejemplos:

```
[1..10]

[2,4..20]

['a'..'e']
```

Operadores en Haskell

Los operadores en haskell son elementos que permiten realizar diferentes operaciones sobre los datos de un programa. Se utilizan para efectuar calculos matematicas, comparar valores, establecer condiciones logicas y manipular listas. Su funcionamiento es parecido al de otros otros lenguajes de programación, aunque haskell presenta algunas diferencias en su sintaxis.

Los operadores se clasifica principalmente en aritmeticos, relacionales y lógicos. Los aritmeticos permiten realizar operaciones como suma, resta, multiplicación y division. Los relacionales comparan dos valores y devuelven un resultado verdadero o falso, mientras que los lógicos permiten combinar condiciones para tomar decisiones dentro de un programa.

También existen operadores especificos para trabajar con listas, como ++, que permite unir dos listas, y :, que agrega un elemento al inicio de una lista.

Ejemplo:

```
8+4 --Resultado: 12
10 `div` 3 Resultado: 3
5 >= 3 Resultado: True
True && False Resultado: False
[1,2] ++ [3,4] Resultado: [1,2,3,4]
```

3. Evaluación perezosa en Haskell

La evaluación perezosa, conocida en inglés como Lazy Evaluation, es una de las características más importantes de Haskell. Consiste en que las expresiones no se calculan inmediatamente, sino únicamente cuando el programa necesita conocer su resultado. Esto permite evitar operaciones innecesarias y trabajar de manera más flexible con los datos.

A diferencia de muchos lenguajes que utilizan una evaluación estricta, Haskell puede definir estructuras de datos sin calcular todos sus elementos desde el principio. Gracias a este comportamiento, es posible utilizar listas infinitas y obtener únicamente los valores necesarios para resolver un problema.

Esta característica también favorece la organización del código, ya que permite separar la definición de los datos de su procesamiento. Sin embargo cuando las expresiones pendientes se acumulan demasiado, pueden aumentar el consumo de memoria.

Ejemplo:

```
take 5 [1..]
```

Resultado: [1,2,3,4,5]

En este ejemplo, [1..] representa una lista infinita de números naturales positivos, pero Haskell solamente genera los primeros cinco elementos porque son los que solicita la función take.